Program Studi Teknik Informatika
Institut Teknologi Sumatera

IF25-11001 ALGORITMA PEMROGRAMAN

Modul Belajar 12

Sorting

Dari papan peringkat lomba sampai menghitung berapa kali dua angka ditukar

Tim Pengajar

Algoritma Pemrograman. Tahun 2026

Komponen	Keterangan
Pertemuan	12 dari 16
Capaian	Sub-CPMK 12.1 dan 12.2, turunan CPMK 0607 dan 0608
Waktu belajar	sekitar 3 jam belajar mandiri, ditambah 100 menit di kelas
Latihan	`02 Latihan/P12 - Sorting - Latihan/`

Peta belajar

Bagian	Isi
0	Tentang modul ini dan cara memakainya
1	Masalah pembuka
Langkah 1	menukar dua nilai
Langkah 2	bubble sort
Langkah 3	trace bubble sort
Langkah 4	sampai terurut penuh
Langkah 5	bubble sort dengan berhenti dini
Langkah 6	selection sort
Langkah 7	trace selection sort
Langkah 8	memilih di antara keduanya
Langkah 9	ringkasan kerja
Langkah 10	dua kesalahan pengurutan
Penutup	Latihan mandiri, bawa ke tugas, rangkuman, cek pemahaman, bacaan lanjut

Tentang modul ini

Modul ini menemani Pertemuan 12. Isinya sama dengan slide, tetapi ditulis lebih pelan supaya bisa dipelajari sendiri, termasuk oleh yang berhalangan hadir.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Diukur di
12.1	Merancang pseudocode bubble sort dan selection sort untuk mengurutkan naik maupun turun, termasuk dua array sejajar	Exit ticket soal 1, Tugas 12, PBL 2, UAS
12.2	Menelusuri isi array sesudah setiap putaran, menghitung perbandingan dan penukaran, dan menjelaskan mengapa keduanya tergolong $O(n^2)$	Exit ticket soal 2, Tugas 12, UAS

Keduanya menurunkan CPMK 0607 dan 0608: menerapkan (0607) dan menjelaskan (0608) konsep dasar algoritma dan sistem komputer untuk menyelesaikan masalah komputasi.

Cara memakai modul ini

Setiap langkah punya kotak merah **Tebak dulu**. Tutup kotak hijau **Jawaban** di bawahnya dengan kertas, tulis dugaan kalian, baru buka. Prosedur penting dijabarkan dalam tiga kolom berdampingan yang dibaca dari kiri ke kanan; sebelum membaca kolom berikutnya, coba tebak dulu langkah apa yang akan datang. Rangkaian setengah jadi lalu mandiri mengurangi bantuan sedikit demi sedikit.

Yang perlu disiapkan

Kertas bergaris, pensil, dan nilai *A* dan *B* dari NIM kalian: *A* digit terakhir (kalau 0, pakai 5) dan *B* digit kedua dari belakang (kalau 0, pakai 3).

Masalah pembuka

PEMBUKA

Himpunan mahasiswa mengadakan lomba pemrograman dengan 200 peserta, dan skor masuk sesuai urutan pengumpulan. Panitia perlu papan peringkat dari skor tertinggi, lalu ingin mencari skor seseorang dengan binary search. Keduanya membutuhkan data terurut. Pertanyaannya: jika hanya boleh menukar dua elemen dalam satu waktu, langkah apa yang menjamin data akhirnya terurut, dan berapa banyak kerja yang dibutuhkan?

Langkah 1: menukar dua nilai

MENUKAR DAN BUBBLE SORT

Penugasan menimpa isi lama. Karena itu menukar dua nilai butuh variabel bantu untuk menyimpan salah

satunya sementara.

TEBAK DULU

a berisi 5 dan b berisi 3. Dewi menulis a ← b lalu b ← a. Apa isi a dan b sekarang?

JAWABAN

Keduanya 3. Baris pertama menimpa 5 sebelum sempat disimpan. Simpan dulu di variabel bantu: `temp` ← `a`, `a` ← `b`, `b` ← `temp`.

Langkah 2: bubble sort

MENUKAR DAN BUBBLE SORT

Bubble sort membandingkan setiap pasangan elemen bersebelahan dan menukarnya jika urutannya salah. Satu putaran membawa nilai terbesar yang tersisa ke ujung kanan. Sesudah i putaran, i elemen paling kanan sudah berada di tempat akhir, sehingga perulangan dalam cukup sampai `N-2-i`. Untuk n elemen dibutuhkan $n - 1$ putaran.

```
UNTUK i ← 0 SAMPAI N-2
  UNTUK j ← 0 SAMPAI N-2-i
    JIKA arr[j] > arr[j+1] MAKA
      temp ← arr[j]
      arr[j] ← arr[j+1]
      arr[j+1] ← temp
    AKHIR JIKA
  AKHIR UNTUK
AKHIR UNTUK
```

Langkah 3: trace bubble sort

MENUKAR DAN BUBBLE SORT

Untuk 5, 3, 8, 1, 4, putaran pertama membandingkan (5,3) dan menukarnya, (5,8) tidak, (8,1) ditukar, (8,4) ditukar. Sesudah putaran pertama isinya 3, 5, 1, 4, 8. Setiap perbandingan memakai isi array saat itu, bukan isi awal.

Langkah 4: sampai terurut penuh

MENUKAR DAN BUBBLE SORT

Banyak perbandingan bubble sort tidak bergantung pada isi data: $(n - 1) + (n - 2) + \cdots + 1 = \frac{n(n-1)}{2}$. Banyak penukaran sama dengan banyak pasangan yang urutannya terbalik.

TEBAK DULU

Lanjutkan dari 3, 5, 1, 4, 8. Tulis isi array sesudah putaran 2, 3, dan 4. Berapa total penukaran dan total perbandingan?

JAWABAN

Putaran 2: 3, 1, 4, 5, 8 (dua tukar). Putaran 3: 1, 3, 4, 5, 8 (satu tukar). Putaran 4: tanpa tukar. Total 6 penukaran dan 4 + 3 + 2 + 1 = 10 perbandingan.

Langkah 5: bubble sort dengan berhenti dini

MENUKAR DAN BUBBLE SORT

Jika satu putaran berlalu tanpa penukaran, data sudah terurut dan sisa putaran tidak perlu dijalankan. Penanda boolean `ditukar` mencatatnya. Perbaikan ini menolong data yang hampir terurut, tetapi kasus terburuk tetap $O(n^2)$.

1 Penanda

Sebelum putaran, `ditukar ← false`. Setiap kali menukar, `ditukar ← true`.

2 Syarat

Perulangan luar diganti `SELAMA i < N-1 DAN ditukar`. Kalau satu putaran tanpa tukar, berhenti.

3 Hasilnya

Data yang sudah terurut cukup satu putaran: $n - 1$ perbandingan. Data terbalik tetap $\frac{n(n-1)}{2}$, jadi terburuknya tetap $O(n^2)$.

UJI DIRI

Pilih jawaban yang benar. (A) 15, (B) 36, (C) 30, (D) 5.

JAWABAN: A

$5 + 4 + 3 + 2 + 1 = 15$. Pilihan B menghitung n^2, pilihan C menghitung $n(n - 1)$ tanpa membagi dua karena lupa perulangan dalam memendek, pilihan D hanya satu putaran.

Langkah 6: selection sort

SELECTION SORT

Selection sort bekerja seperti misi kartu kedua. Pada putaran ke-i, perulangan dalam mencari indeks nilai terkecil di bagian yang belum terurut, lalu nilai itu ditukar ke posisi i. Penukaran terjadi paling banyak sekali per putaran, sehingga totalnya paling banyak $n - 1$. Banyak perbandingannya tetap $\frac{n(n-1)}{2}$.

```
UNTUK i ← 0 SAMPAI N-2
  minIdx ← i
  UNTUK j ← i+1 SAMPAI N-1
    JIKA arr[j] < arr[minIdx] MAKA
      minIdx ← j
    AKHIR JIKA
  AKHIR UNTUK
  JIKA minIdx ≠ i MAKA
    tukar arr[i] dan arr[minIdx] // pakai temp
  AKHIR JIKA
AKHIR UNTUK
```

Langkah 7: trace selection sort

SELECTION SORT

Untuk 5, 3, 8, 1, 4, putaran 0 menemukan 1 di indeks 3 dan menukarnya dengan 5, menghasilkan 1, 3, 8, 5, 4. Putaran 1 tidak menukar karena 3 sudah di tempatnya. Putaran 2 menukar 8 dengan 4. Putaran 3 tidak menukar. Hasil 1, 3, 4, 5, 8 dengan dua penukaran, padahal bubble sort butuh enam.

Langkah 8: memilih di antara keduanya

SELECTION SORT

Pada data yang hampir terurut, bubble sort dengan berhenti dini lebih hemat perbandingan. Selection sort selalu memeriksa seluruh sisa data.

TEBAK DULU

Data 1, 2, 3, 5, 4. Berapa perbandingan dan penukaran bubble sort dengan berhenti dini, dan berapa untuk selection sort?

JAWABAN

Bubble: putaran 1 menukar 5 dan 4 (4 perbandingan), putaran 2 tanpa tukar (3 perbandingan) lalu berhenti; total 7 perbandingan, 1 penukaran. Selection: tetap 10 perbandingan, 1 penukaran. Selection tidak bisa tahu bahwa data sudah hampir terurut.

Langkah 9: ringkasan kerja

SELECTION SORT

Kedua algoritma tergolong $O(n^2)$ karena banyak perbandingannya sebanding dengan n^2. Bedanya ada pada penukaran: bubble sort menukar sebanyak pasangan yang terbalik, selection sort paling banyak $n - 1$ kali. Bubble sort stabil, artinya dua nilai kembar tetap dalam urutan asalnya; selection sort tidak menjamin itu.

Setengah jadi

Selection sort urut **turun**.

```
UNTUK i ← 0 SAMPAI N-2
  maxIdx ← ____
  UNTUK j ← ____ SAMPAI N-1
    JIKA arr[j] ____ arr[maxIdx] MAKA
      maxIdx ← ____
  tukar arr[i] dan arr[____]
```

Mandiri

Papan peringkat: `nama[N]` dan `skor[N]` sejajar. Tulis bubble sort yang mengurutkan skor turun dan membawa nama ikut pindah.

JAWABAN

Setengah jadi: `i`, `i+1`, `>`, `j`, `maxIdx`. Mandiri: pembanding `skor[j] < skor[j+1]`, lalu dua penukaran dalam blok JIKA yang sama, satu untuk skor dan satu untuk nama, masing-masing dengan variabel bantunya.

UJI DIRI

Pilih jawaban yang benar. (A) 1, 2, 6, 4, 3, (B) 2, 4, 1, 3, 6, (C) 1, 4, 2, 6, 3, (D) 1, 2, 3, 4, 6.

JAWABAN: A

Terkecil 1 di indeks 3, ditukar dengan 4. Pilihan B adalah hasil putaran pertama bubble sort. Pilihan C menggeser semua elemen, bukan menukar. Pilihan D sudah terurut penuh.

Langkah 10: dua kesalahan pengurutan

SELECTION SORT

Dua kesalahan pengurutan paling sering: batas perulangan dalam yang membuat `arr[j+1]` keluar dari array, dan array sejajar yang hanya salah satunya ditukar sehingga pasangan data tercerai.

Latihan mandiri

Kerjakan lembar latihan di folder `02 Latihan`. Urutannya disusun dari mengingat ke menganalisis.

Tingkat	Waktu	Yang dilatih
1, Dasar	5′	trace putaran pertama kedua algoritma
2, Menengah	8′	pengurutan lengkap dan array sejajar
3, Tantangan	7′	memperbaiki bubble sort
Pengayaan	sisa	bubble sort dengan berhenti dini

Kesalahan yang sering muncul

Yang ditulis	Masalahnya	Obatnya
`a ← b, b ← a`	Satu nilai hilang	Pakai `temp`
Perulangan dalam sampai `N-1`	Membaca `arr[N]`	Sampai `N-2-i`
`minIdx ← arr[j]`	Menyimpan nilai, bukan posisi	`minIdx ← j`
Hanya skor yang ditukar	Nama tercerai dari skornya	Tukar semua array sejajar di blok yang sama

Bawa ke tugas

Tugas 12 dikerjakan di kertas: untuk `arr` = 12, 5, 18, 3, 9, 15, 1, trace bubble sort dan selection sort sampai terurut, hitung perbandingan dan penukaran masing-masing, lalu jelaskan mana yang lebih hemat untuk

data ini; tulis selection sort urut turun. PBL 2 Data Analyzer dikumpulkan hari ini.

Rangkuman

Selalu dengan variabel bantu: tiga penugasan. Bubble menukar tetangga, terbesar ke kanan. Selection memilih terkecil, tukar sekali per putaran. Keduanya $\frac{n(n-1)}{2}$ perbandingan, $O(n^2)$. Penukaran: bubble sebanyak pasangan terbalik, selection paling banyak $n - 1$.

Cek pemahaman

Jawab tanpa membuka modul atau slide. Mengingat kembali tanpa melihat jauh lebih menguatkan ingatan daripada membaca ulang.

No	Soal
1	Berapa perbandingan bubble sort tanpa berhenti dini untuk 10 data?
2	Selection sort pada 10 data, paling banyak berapa penukaran?
3	Apa arti stabil?
4	Kapan bubble sort dengan berhenti dini hanya butuh satu putaran?
5	Mengapa keduanya disebut $O(n^2)$?

KUNCI CEK PEMAHAMAN

(1) 45. (2) 9. (3) Nilai kembar tetap dalam urutan asalnya. (4) Saat data sudah terurut. (5) Banyak perbandingannya $\frac{n(n-1)}{2}$, sebanding dengan n^2.

Bacaan lanjut

Gaddis, T. *Starting Out with C++*, Pearson. Deitel, P. dan Deitel, H. *C++ How to Program*, Pearson. Cormen dkk. *Introduction to Algorithms*, MIT Press. Visualisasi interaktif di `visualgo.net`.